

BIO-INSPIRED ARTIFICIAL INTELLIGENCE

Swarm Intelligence II

Dr. Giovanni Iacca

giovanni.iacca@unitn.it



UNIVERSITY OF TRENTO - Italy

**Information Engineering
and Computer Science Department**

Ant Colony Optimization



ANT COLONY OPTIMIZATION

EMERGENT PROBLEM SOLVING IN ANTS

- Regulation of nest temperature (in some species, within a range of ~ 1 Celsius degree)
- Forming bridges
- Building and protecting nest
- Sorting brood, corpses, garbage and food items
- Cooperating in carrying large items
- Emigration of a colony
- Finding shortest route from nest to food through stigmergy

These are all examples of an emergent
swarm behavior

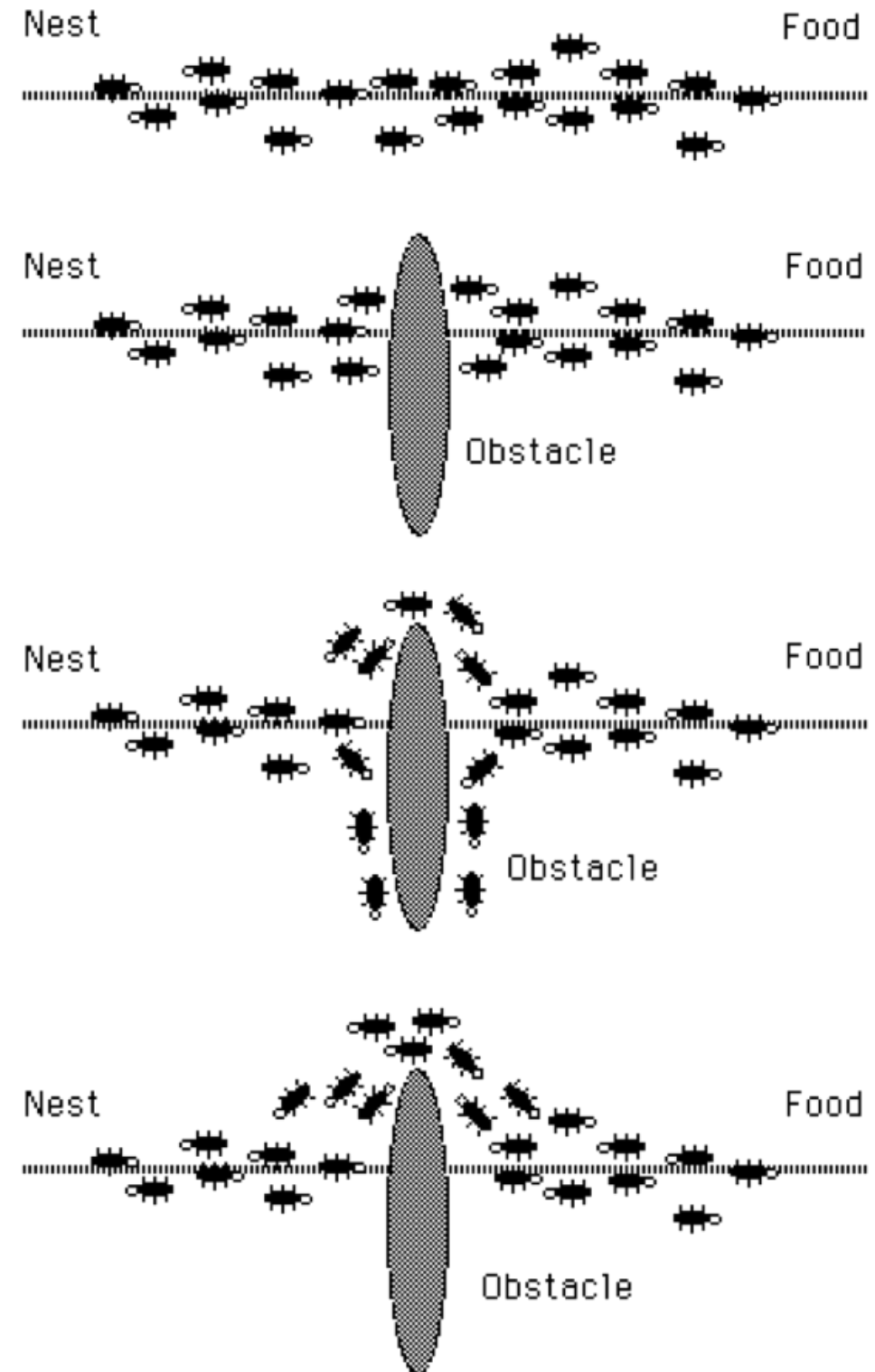
Beyond what any individual can do!



ANT COLONY OPTIMIZATION

WHAT IS STIGMERGY?

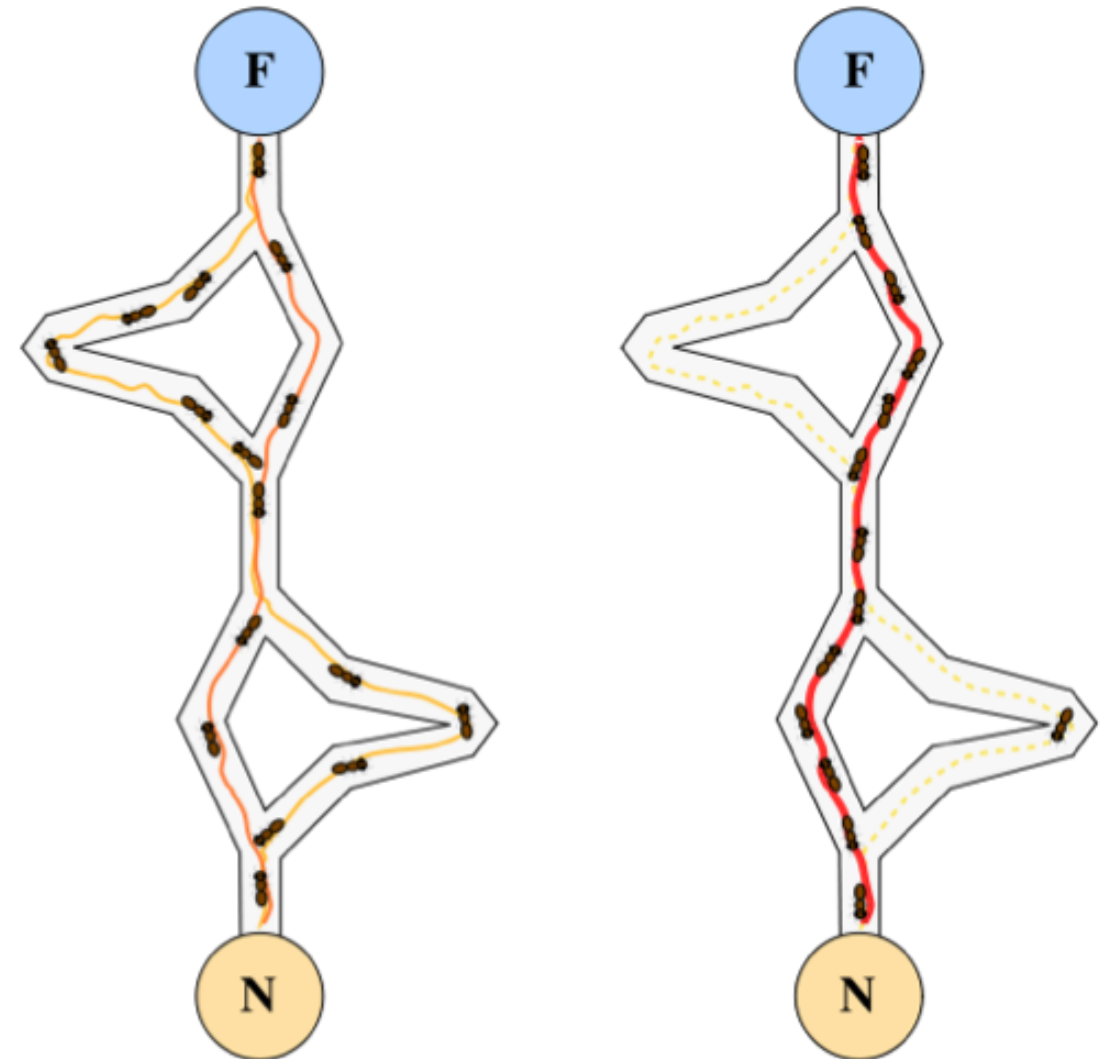
- The term indicates a form of communication among individuals through modification of the environment.
- For example, some ants leave a chemical (pheromone) trail behind, to trace a path. The chemical decays over time.
- This allows other ants to find the path between the food and the nest. It also allows ants to find the shortest path among alternative paths.



ANT COLONY OPTIMIZATION

WHAT IS STIGMERGY?

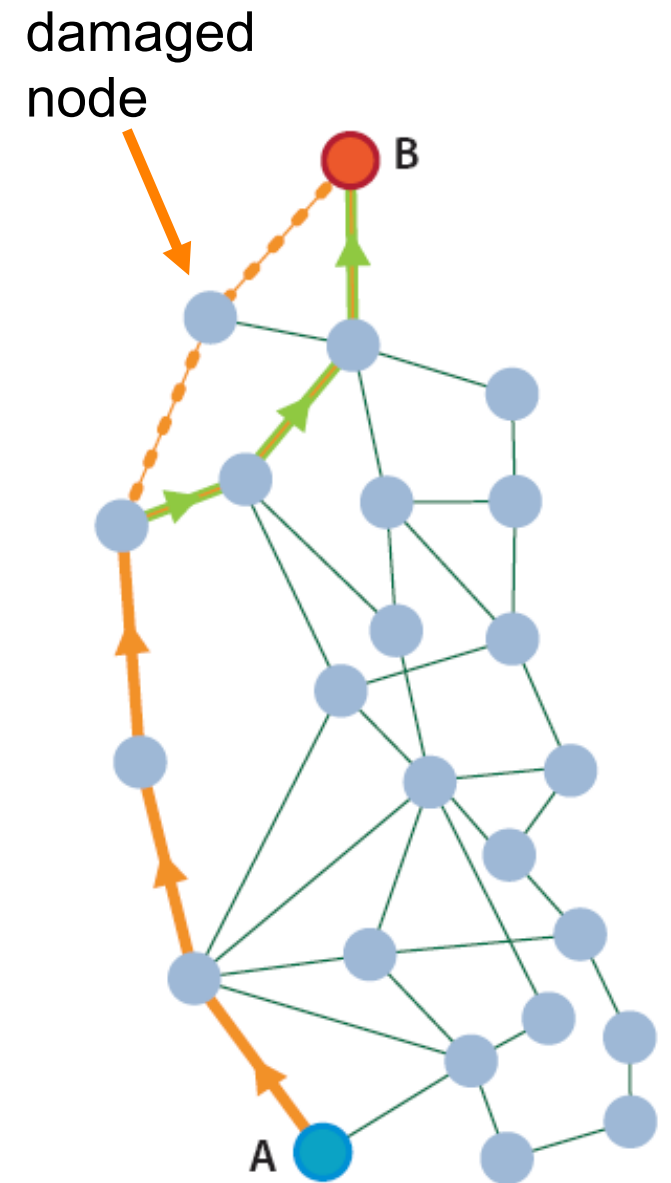
- Ants follow the path with the highest pheromone concentration.
- Without pheromone, there's an equal probability of choosing a short or a long path.
- Shorter paths allow a higher number of passages and therefore the pheromone level tends to be higher (and decay more slowly).
- Ants will increasingly tend to choose shorter path (a sort of reinforcement mechanism based on a positive feedback loop, or **auto-catalytic behavior**).



ANT COLONY OPTIMIZATION

GENERALITIES

- **Ant Colony Optimization (ACO)** is an algorithm developed by Dorigo et al. in 1992, inspired upon stigmergic communication to find the shortest path in a network. The original implementation was called **Ant System (AS)**.
- Typically ACO assumes an optimization problem represented as a graph problem: typical examples are combinatorial problems over networks (e.g. routing), and in general **any problem that can be described as Travel Salesman Problem (TSP)**, e.g. VRPTW
- Modern ACO algorithms include hybridizations with other metaheuristics & local search, self-adaptation, and are extended to continuous and multi-objective optimization (a lot of research!)
- Today, ACO is used in many real-worlds problems in logistics & scheduling, e.g. by British Telecom, MCI Worldcom, Barilla, Migros, etc.



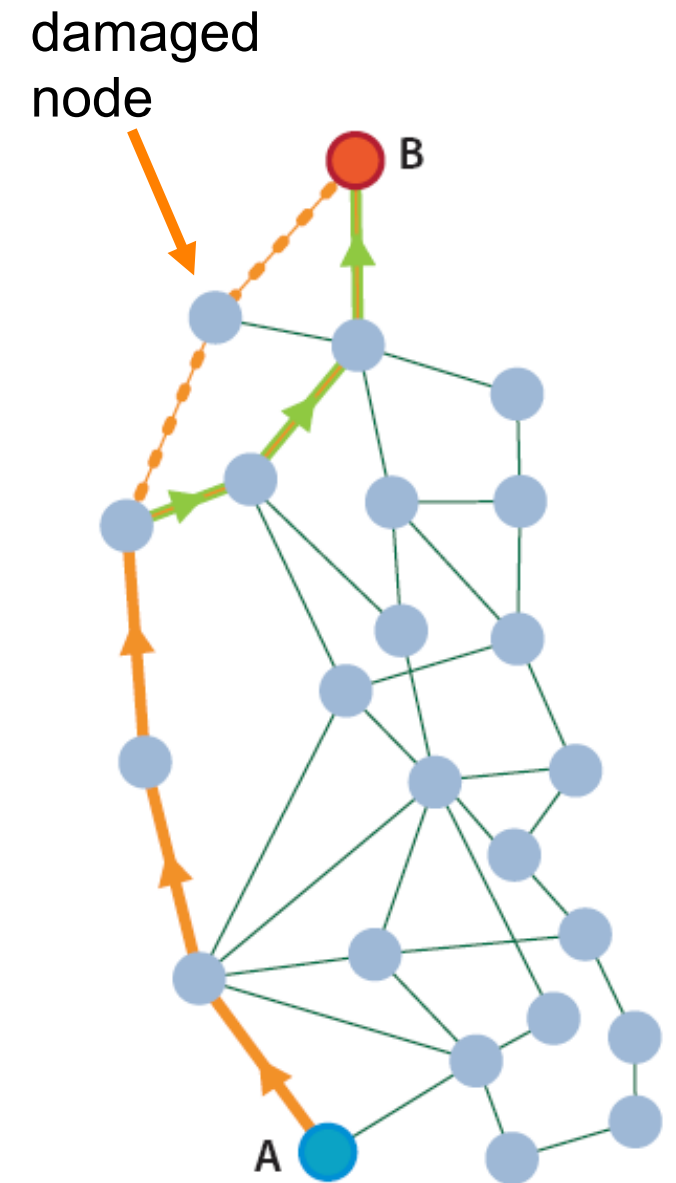
ANT COLONY OPTIMIZATION

ADVANTAGES

- Dynamic rerouting through shortest path naturally occurs if one node is broken.
- Positive feedback accounts for rapid discovery of good solutions.
- Distributed (inherently parallel) computation with stochasticity avoids premature convergence.
- A greedy heuristic helps find acceptable solutions in the early stages of the search process.
- The collective interaction of a population of agents helps solving complex problems.

DISADVANTAGES

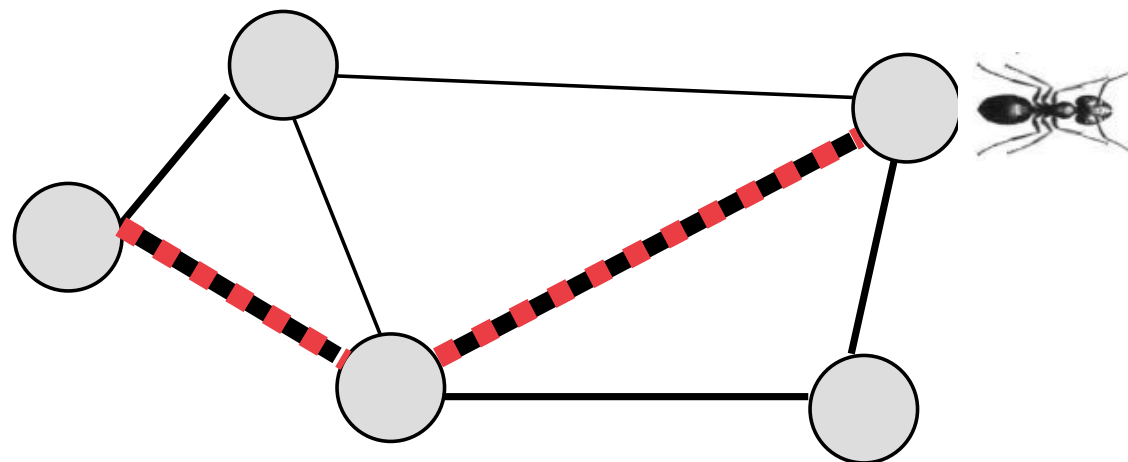
- Sometimes slower than other metaheuristics (esp. on large problems).
- Sometimes, the absence of a central coordination can be inefficient.



ANT COLONY OPTIMIZATION

GENERALITIES

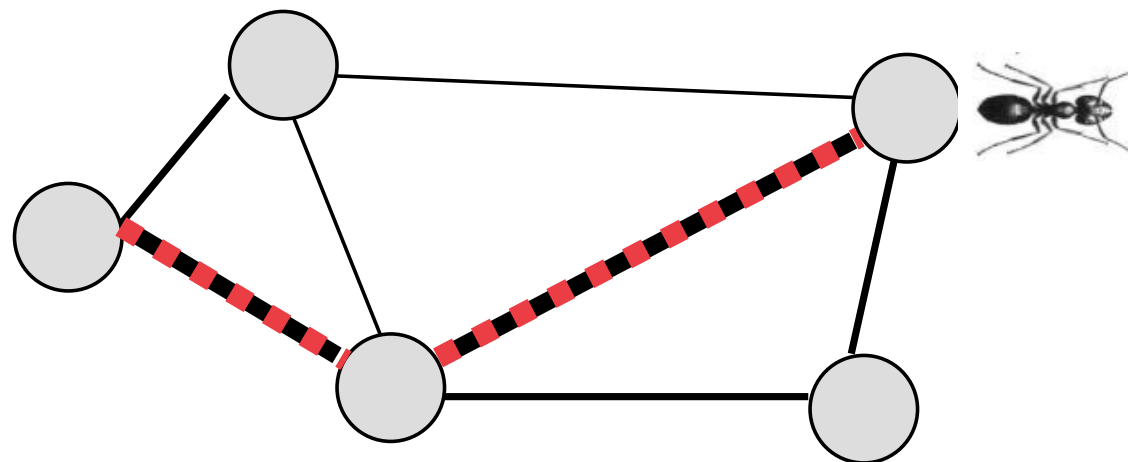
- Ants are simple stimulus-response agents that stochastically and iteratively construct a solution to a graph problem.
 - ▶ This constructive aspect is what distinguishes ACO algorithms from other metaheuristics!
- Each ant moves on a graph, and chooses where to go based on pheromone strength and a problem-dependent heuristic (typically, distance)
 - ▶ A **probabilistic transition rule** is used to encourage the choice of edges with high pheromone levels and short distances.
 - ▶ Usually, each ant keeps a “path memory” (i.e. tabu moves prevent visiting the same node twice)
- The total ant's path represents a specific candidate solution.



ANT COLONY OPTIMIZATION

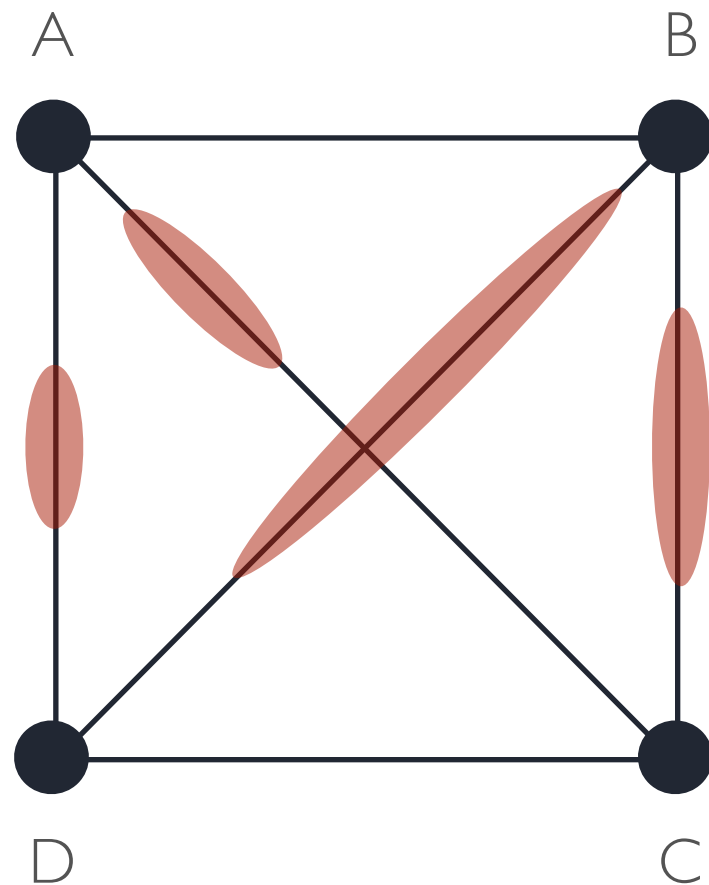
GENERALITIES

- When an ant completes a solution, pheromone is laid on its path, according to quality of solution.
- Usually, the increase of pheromone level for the edges of a certain path is inversely proportional to the total length of the path to which those edges belong.
 - ▶ The result is that edges that belong to **short paths** receive a greater amount of pheromone.
- The pheromone level on each edge is then reduced by **evaporation**.
- This pheromone trail affects behavior of other ants by stigmergy.

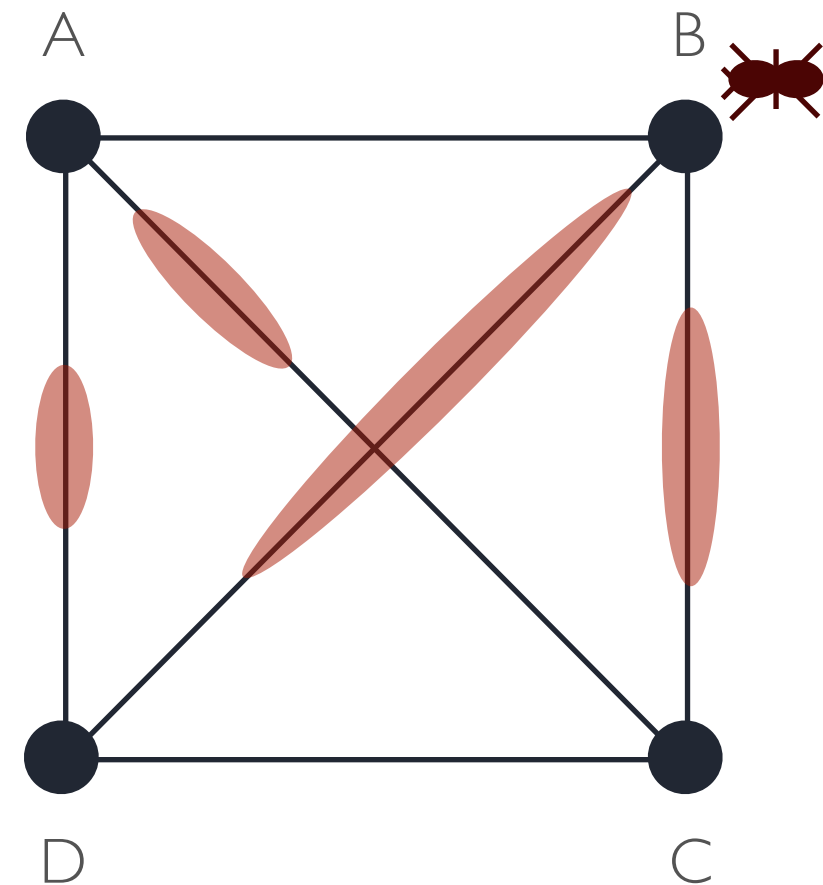


ANT COLONY OPTIMIZATION

EXAMPLE: 4-CITY TSP



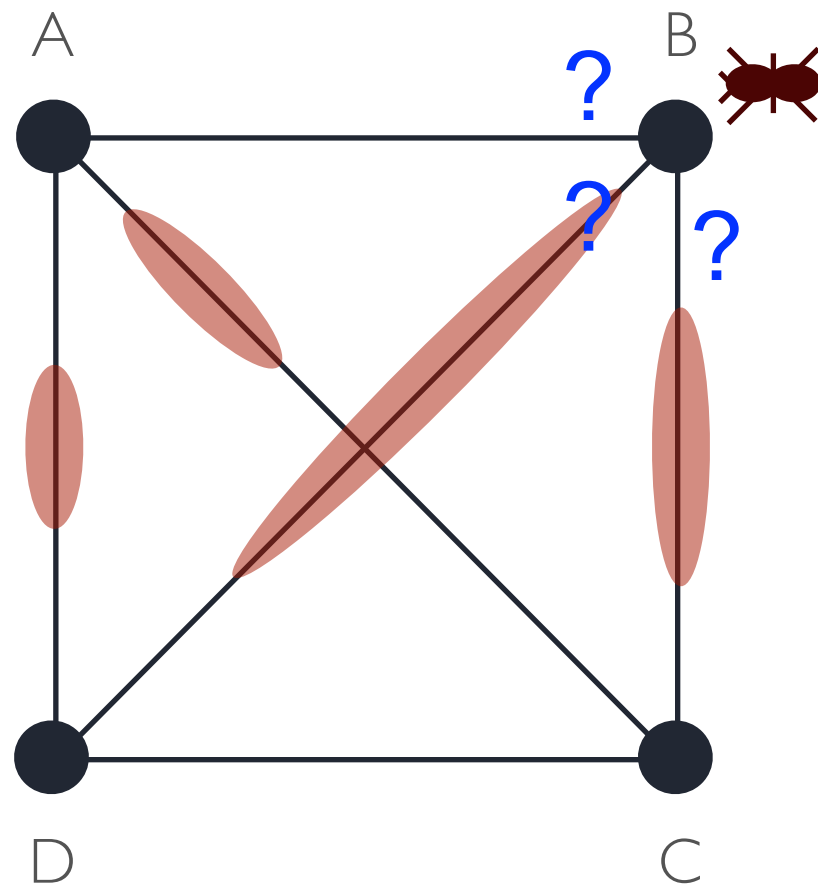
Initially, random levels of pheromone are scattered on the edges.



An ant is placed at a random node.

ANT COLONY OPTIMIZATION

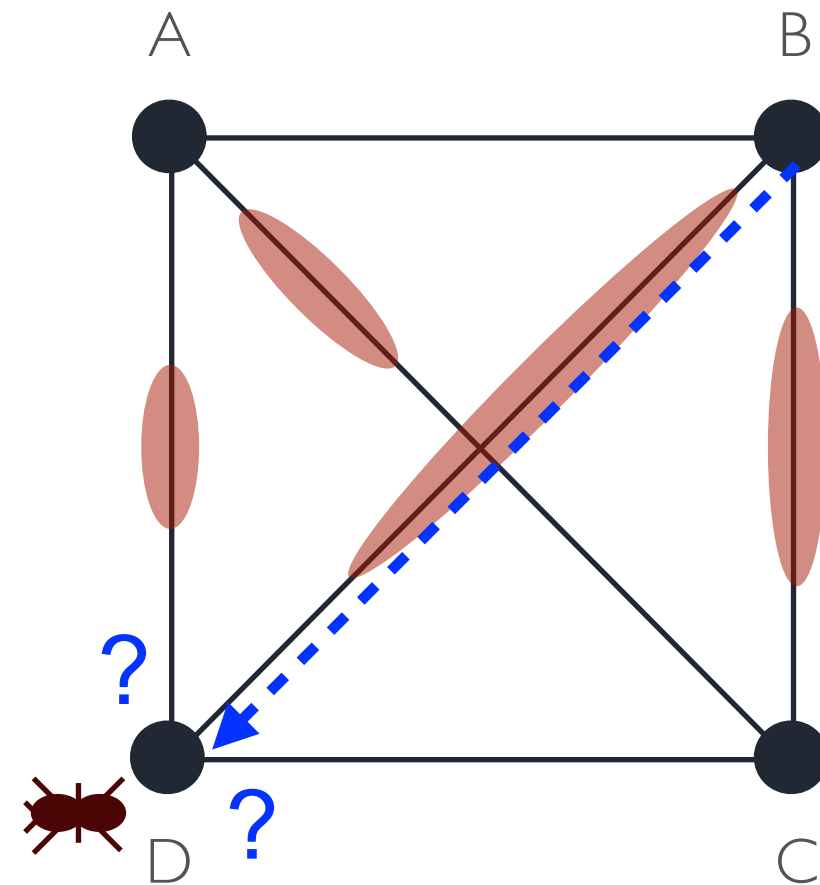
EXAMPLE: 4-CITY TSP



The ant decides where to go from that node, based on probabilities calculated from:

- pheromone strengths
- next-hop distances

Suppose it chooses BD.

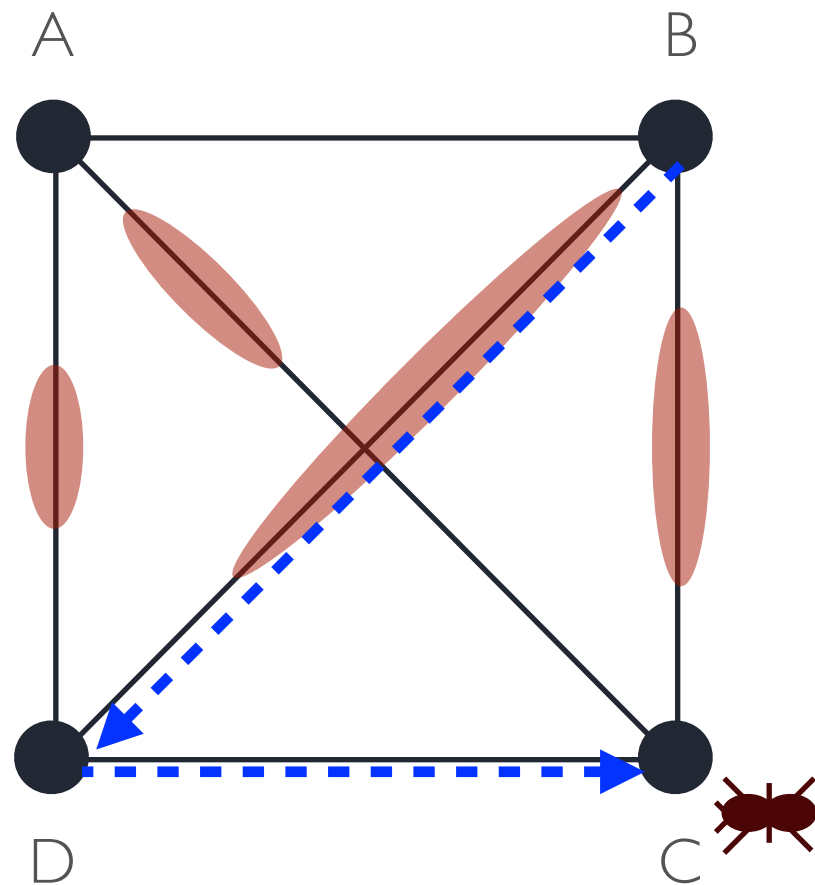


The ant is now at D, and has a “path memory”: {B, D}, such that it cannot visit B or D again. Again, it decides the next hop (from those allowed) based on pheromone strength and distance.

Suppose it chooses DC.

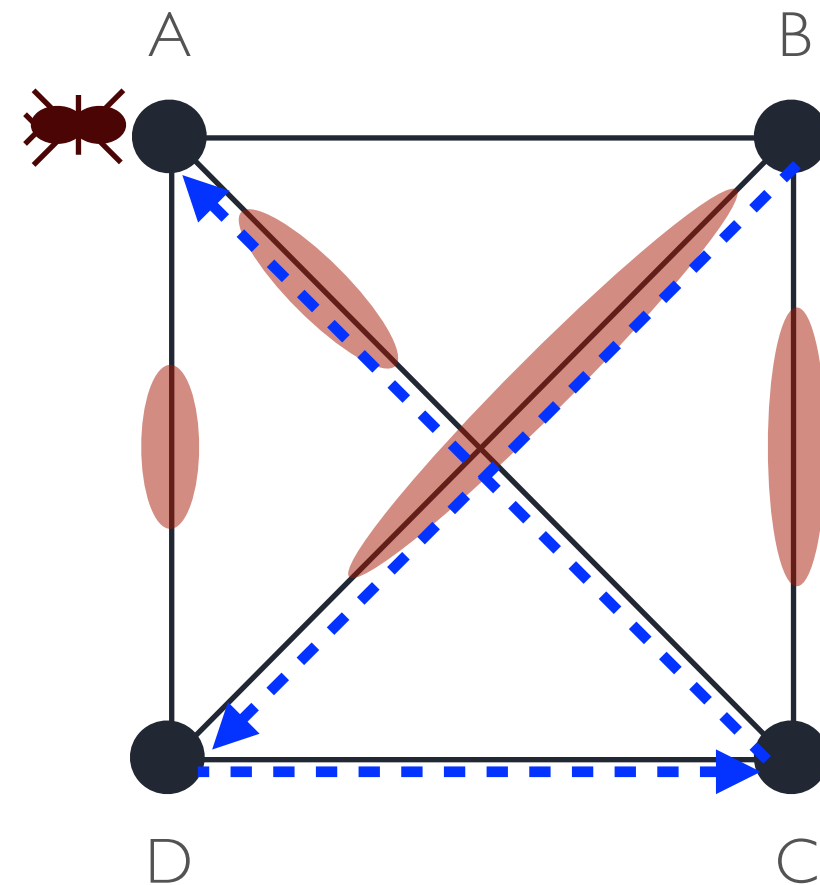
ANT COLONY OPTIMIZATION

EXAMPLE: 4-CITY TSP



The ant is now at C, and has a “path memory”: {B, D, C}.

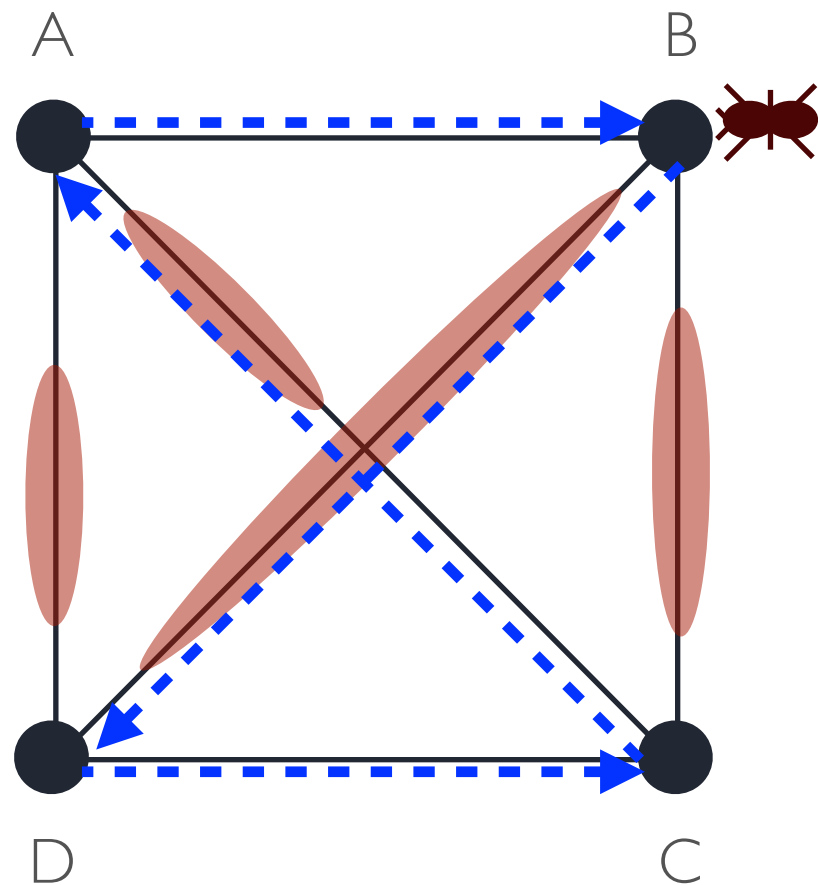
There is only one place it can go now: CA.



So, it has nearly finished his path, having gone over the links: BD, DC, and CA.

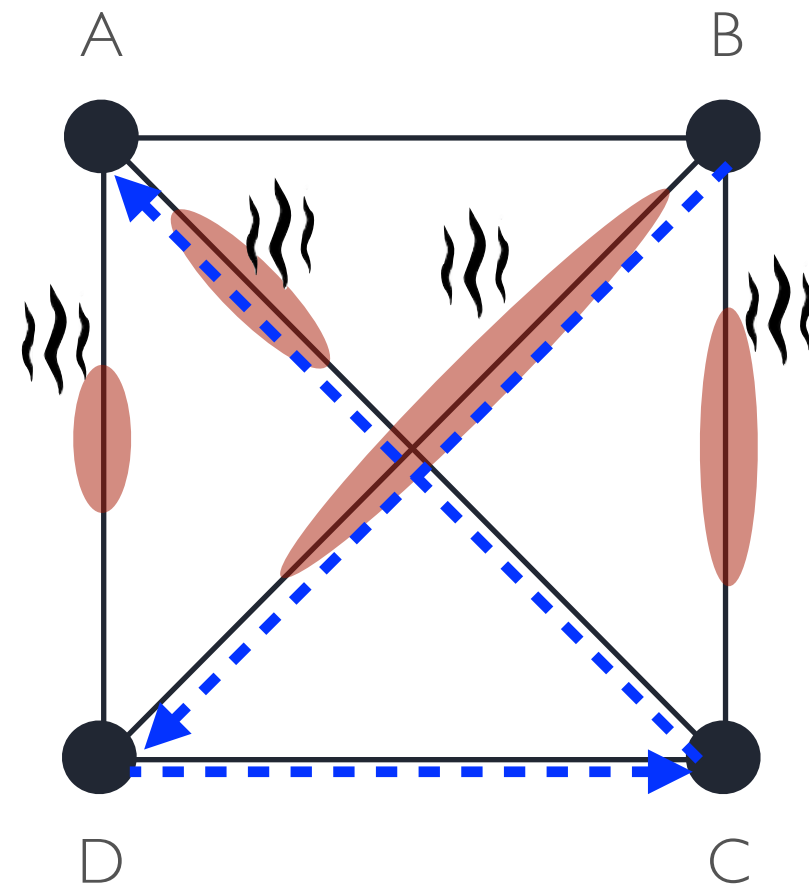
ANT COLONY OPTIMIZATION

EXAMPLE: 4-CITY TSP



AB is added to complete the round trip.

Now, pheromone on the path is increased, based on the fitness (total length) of that path.



Next, pheromone everywhere is decreased a little, to model decay of trail strength over time.

And we start again, with another ant in a random position...

ANT COLONY OPTIMIZATION

ALGORITHMIC DETAILS: TRANSITION RULE

- **How do artificial ants decide which path to take?**

- At each iteration, each ant incrementally constructs a path (a solution)
- At each step t of the iteration, each ant k samples a random number q in $[0,1]$. If q is smaller than a threshold q_0 , then it chooses the edge with the largest amount of pheromone τ (or the smallest value of the problem-dependent heuristic, usually shortest length η), otherwise use a probabilistic transition rule:

$$p_{ij}^k(t) = \frac{[\tau_{ij}(t)]^\alpha \cdot [\eta_{ij}]^\beta}{\sum_{l \in \mathcal{N}_i^k} [\tau_{il}(t)]^\alpha \cdot [\eta_{il}]^\beta} \quad \text{if } j \in \mathcal{N}_i^k$$

i.e., the ant “moves” from node i to node j with probability proportional to the amount of pheromone τ and length η of the edge i - j , relative to all other nodes connected to i .

- Loops are removed once the destination node has been reached (alternatively, ants use path memory)
- α and β influence the algorithms behavior:
 - $\alpha=0 \rightarrow$ the action is based solely on edge length information
 - $\beta=0 \rightarrow$ the action is based solely on pheromone accumulation information

ANT COLONY OPTIMIZATION

ALGORITHMIC DETAILS: PHEROMONE UPDATE (ORIGINAL AS)

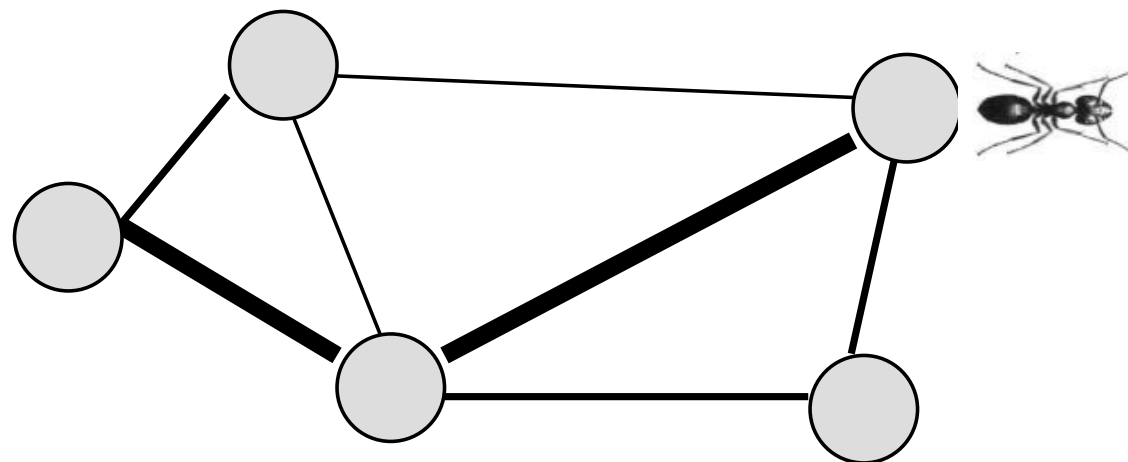
- Typically, the pheromone level of each edge is updated as follows:

$$\tau_{ij}(t+1) = (1 - \rho) \cdot \tau_{ij}(t) + \sum_{k=1}^m \Delta\tau_{ij}^k(t) \quad \forall(i, j)$$

where:

$$\Delta\tau_{ij}^k(t) = \begin{cases} 1/L^k(t) & \text{if arc } (i, j) \text{ is used by ant } k \\ 0 & \text{otherwise} \end{cases}$$

in other words, the pheromone on an edge decays linearly with time (depending on ρ) but is increased by the transition of all ants on that edge, rescaled by the length $L^k(t)$ of each ant's path.



ANT COLONY OPTIMIZATION

ALGORITHMIC DETAILS: PHEROMONE UPDATE (MODERN ACO)

- **Local pheromone level**

The pheromone level of each edge visited by an ant is decreased by a fraction $(1-\rho)$ of its current level and increased by a fraction ρ of the initial level $\tau(0)$:

$$\tau_{ij}(t+1) = (1-\rho) \cdot \tau_{ij}(t) + \rho \cdot \tau_{ij}(0)$$

- **Global pheromone level**

When all ants have completed their paths, the length L of the shortest path is found and only the pheromone levels of the edges belonging to the shortest path are updated, with a rule depending (in inverse proportion) on the path length:

$$\tau_{ij}(t+1) = (1-\rho) \cdot \tau_{ij}(t) + \rho/L$$

ANT COLONY OPTIMIZATION

APPLICATIONS

- ACO is naturally applicable to any sequencing problem, or indeed *any* problem
- All we need is some way to represent solutions to the problem as paths in a graph

$t = 0$ and initialize all parameters

Place all ants, $k = 1, \dots, m$ and Initialize pheromone on each link;

repeat

for *each ant* $k = 1, \dots, m$ **do**

 Construct a path, $x^k(t)$;

 Compute $f(x^k(t))$;

end

for *each link* (i, j) **do**

 Apply pheromone evaporation, then update pheromone;

$\tau_{ij}(t + 1) = \tau_{ij}(t)$;

end

$t = t + 1$;

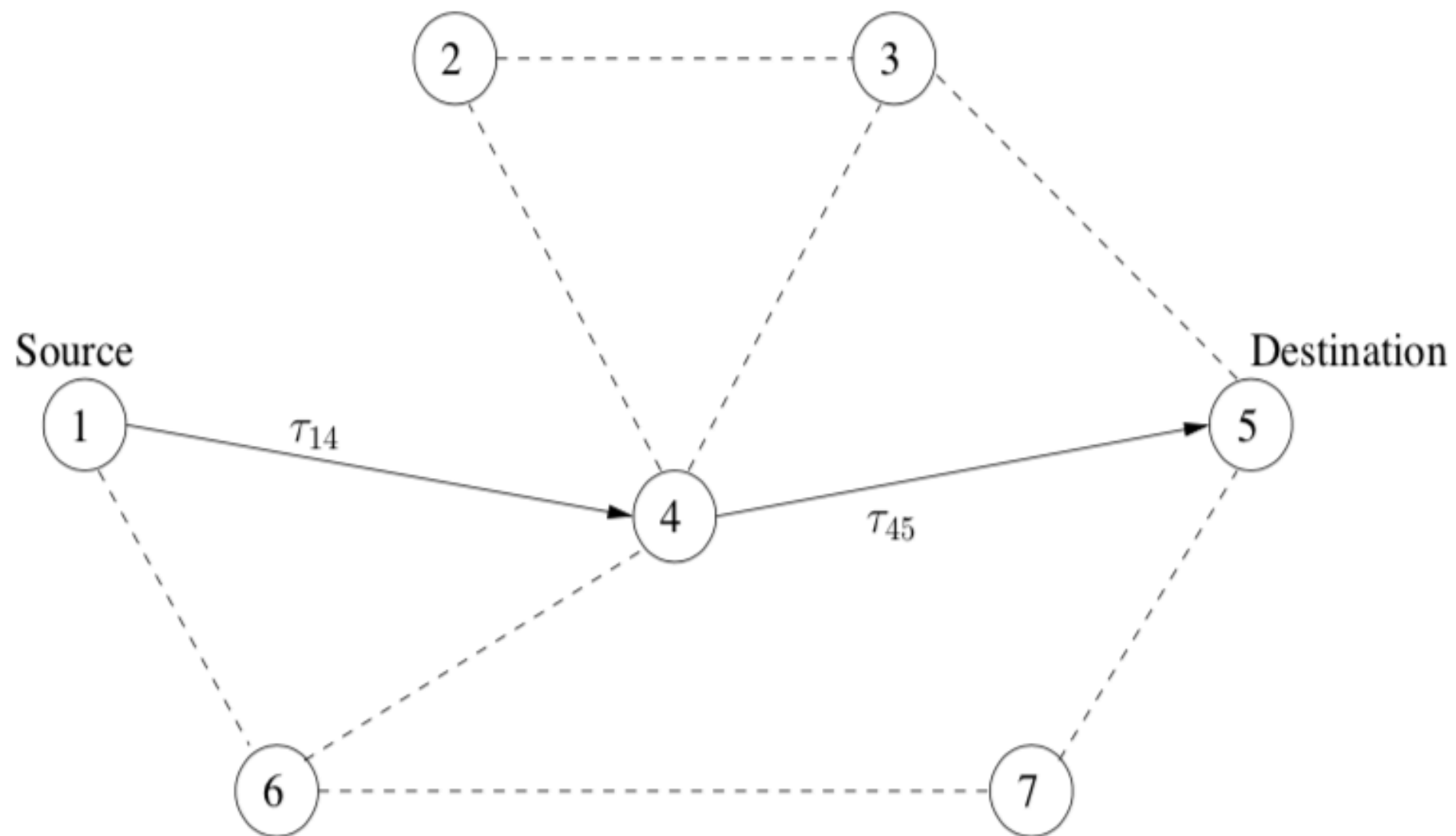
until *stopping condition is true* ;

Return $x^k(t) : f(x^k(t)) = \min_{k'=1, \dots, n_k} \{f(x^{k'}(t))\}$;

ANT COLONY OPTIMIZATION

EXAMPLE: SHORTEST PATH

Consider the general problem of finding a shortest path between two nodes:



ANT COLONY OPTIMIZATION

EXAMPLE: SHORTEST PATH

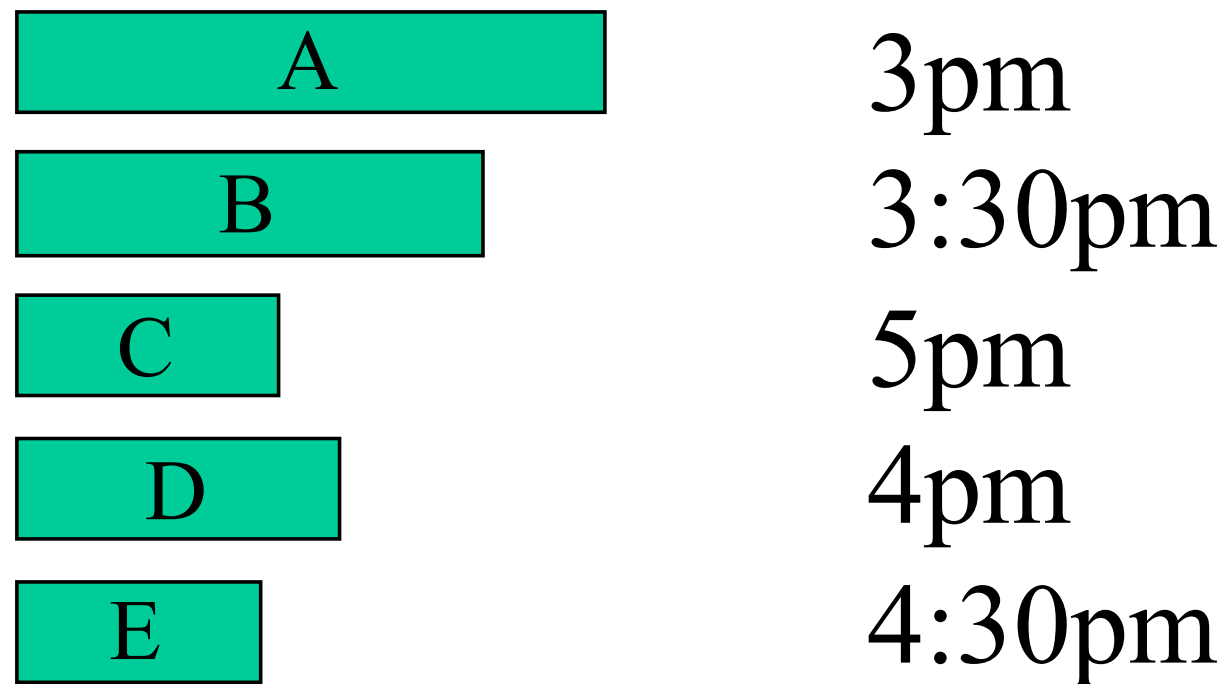
Algorithm parametrization:

- m ants (e.g. $m = 100$) are generated, initially distributed on random nodes
- Typically, the initial pheromone level is equal for all edges and inversely proportional to number of nodes in the graph (n) times the estimated length (L^*) of the optimal path: $\tau(0) = 1/(n \cdot L^*)$
- Typical parameter setting:
 - relative importance of pheromone and edge length $\alpha=1$ and $\beta=2$
 - exploration threshold $q_0=0.9$
 - pheromone update rate $\rho=0.1$

ANT COLONY OPTIMIZATION

EXAMPLE: SINGLE MACHINE SCHEDULING WITH DUE-DATES

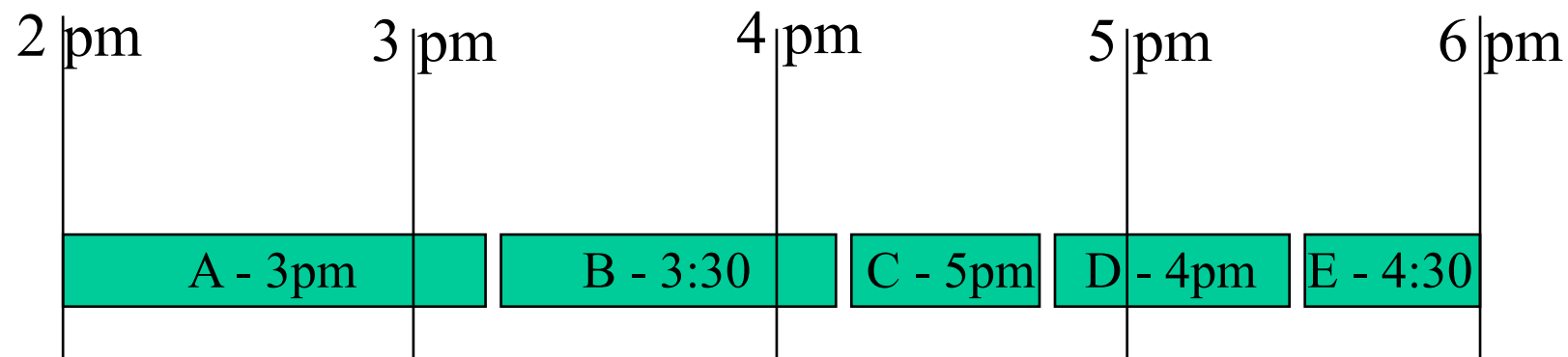
A certain number of jobs have to be done; their length represents the time they will take. Each job has a due date, when it needs to be finished.



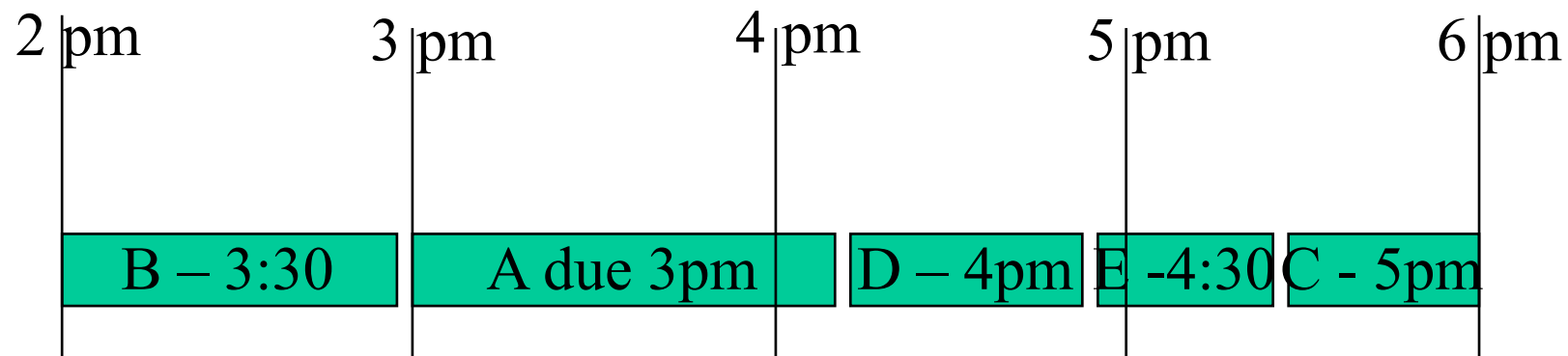
ANT COLONY OPTIMIZATION

EXAMPLE: SINGLE MACHINE SCHEDULING WITH DUE-DATES

Some possible schedules:



A is 10min late
B is 40min late
C is 20min early (lateness = 0)
D is 90min late
E is 90min late



A is 70min late
B is 30min early (0 lateness)
C is 60min late
D is 50min late
E is 50min late

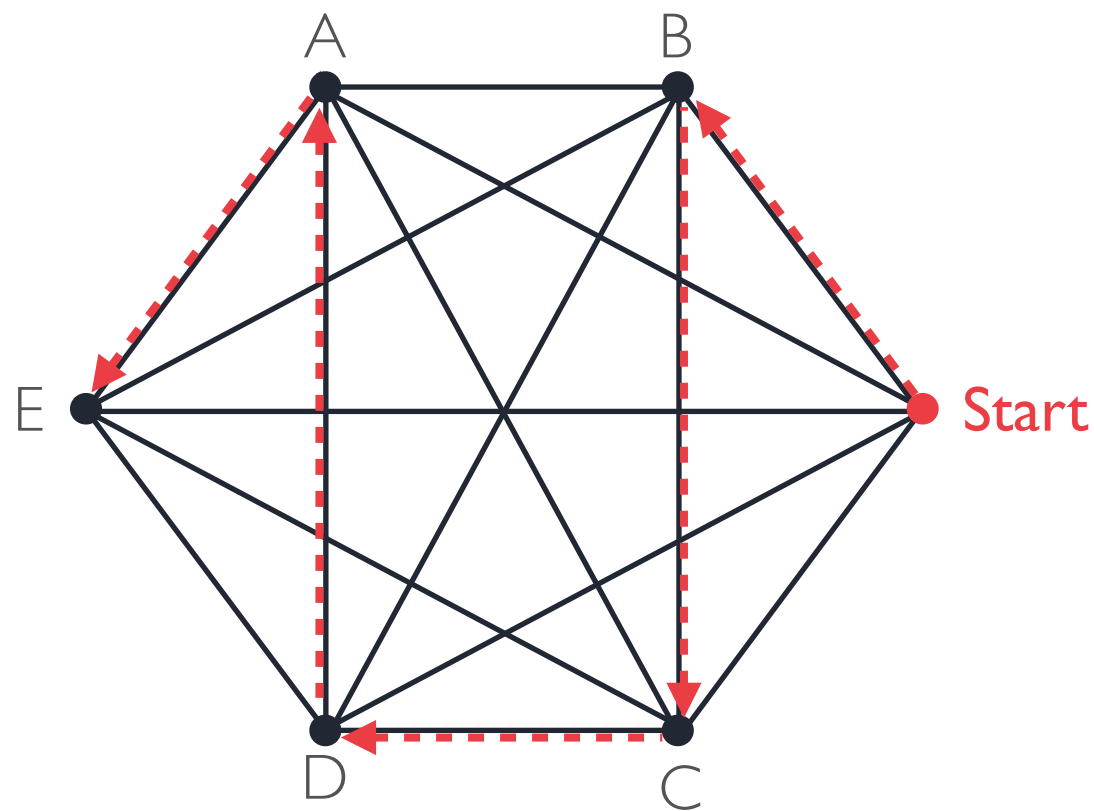
Possible fitness functions might be:

- average lateness (46 min for both solutions)
- max lateness (90 min for the first solution, 70 min for the second one)

ANT COLONY OPTIMIZATION

EXAMPLE: SINGLE MACHINE SCHEDULING WITH DUE-DATES

- How to apply ACO: just like with the TSP, each ant finds paths in a graph, where each job is a node
- In this case, no need to return to a start node (path is complete when every node is visited)

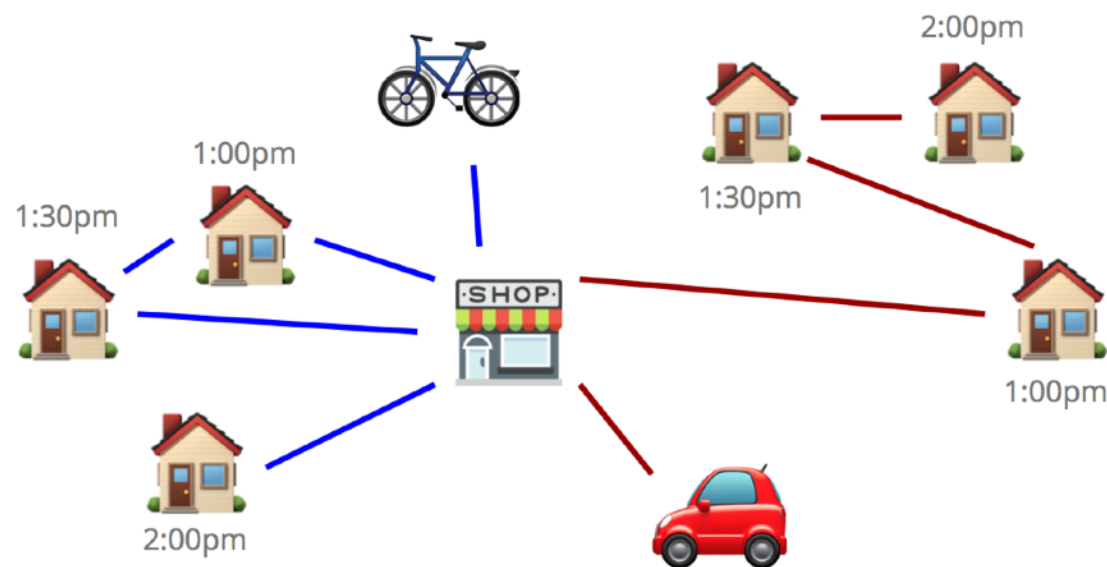


- ▶ Initially, random levels of pheromone are scattered on the edges
- ▶ An ant starts at a “virtual” start node (such that the first edge it chooses defines the first task to schedule on the machine)
- ▶ The ant uses a transition rule to take one step at a time, biased by the pheromone levels and a heuristic score, each time choosing the next machine to schedule
- ▶ Duplicates are avoided by introducing *tabu* moves

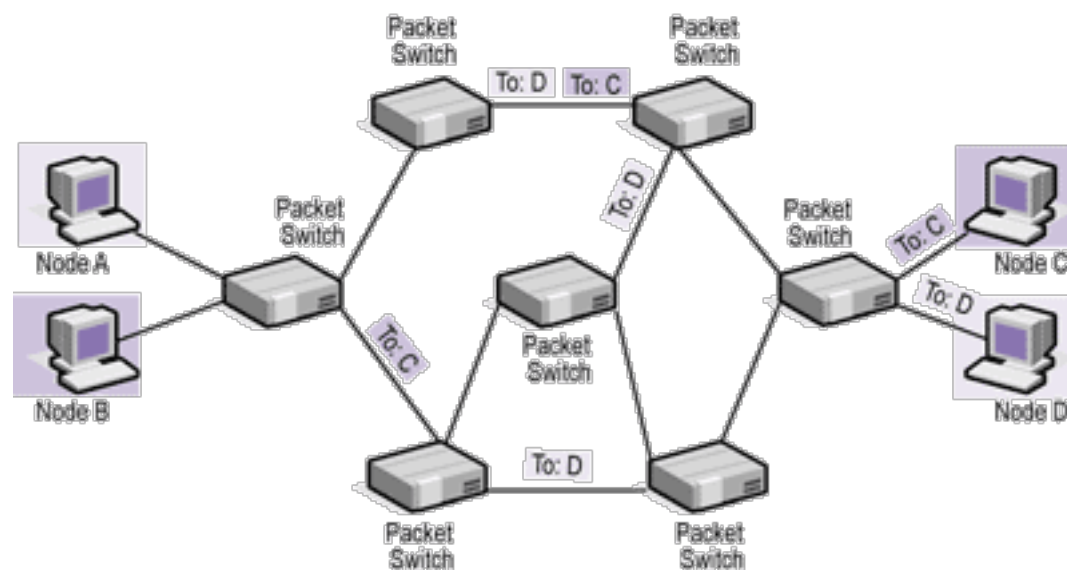
Q: What heuristic score could we use in this case?

ANT COLONY OPTIMIZATION

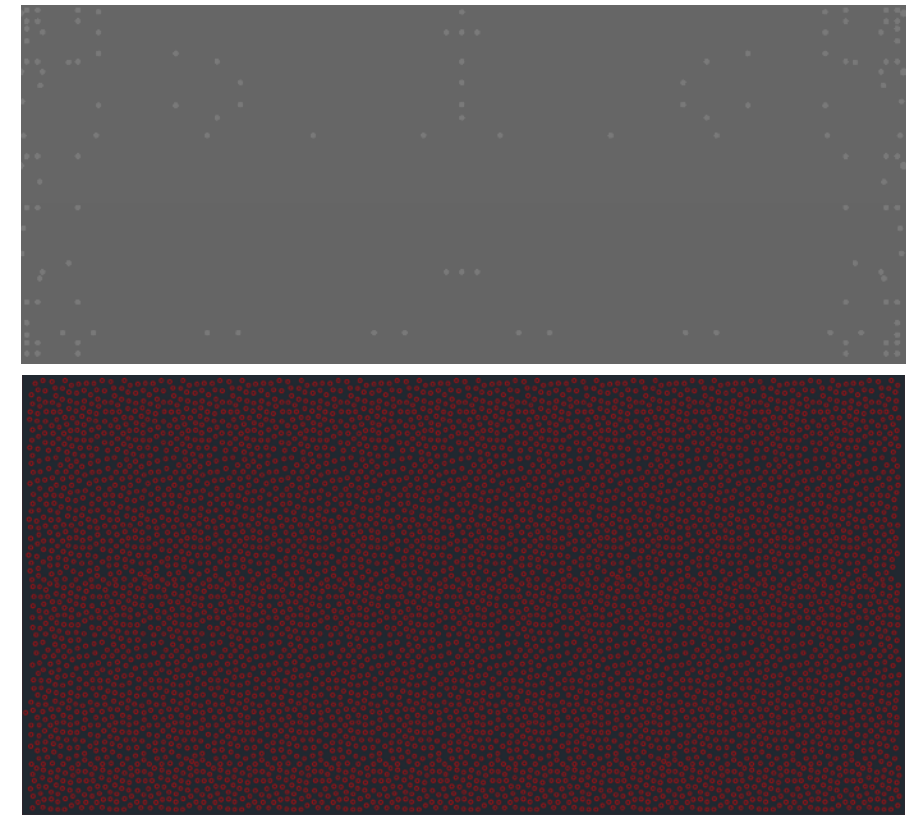
OTHER APPLICATIONS



CVRPTWMT: Capacitated Vehicle Routing Problem with Time Windows and Multiple Trips



Network packet routing



Drilling wooden boards / iron sheets

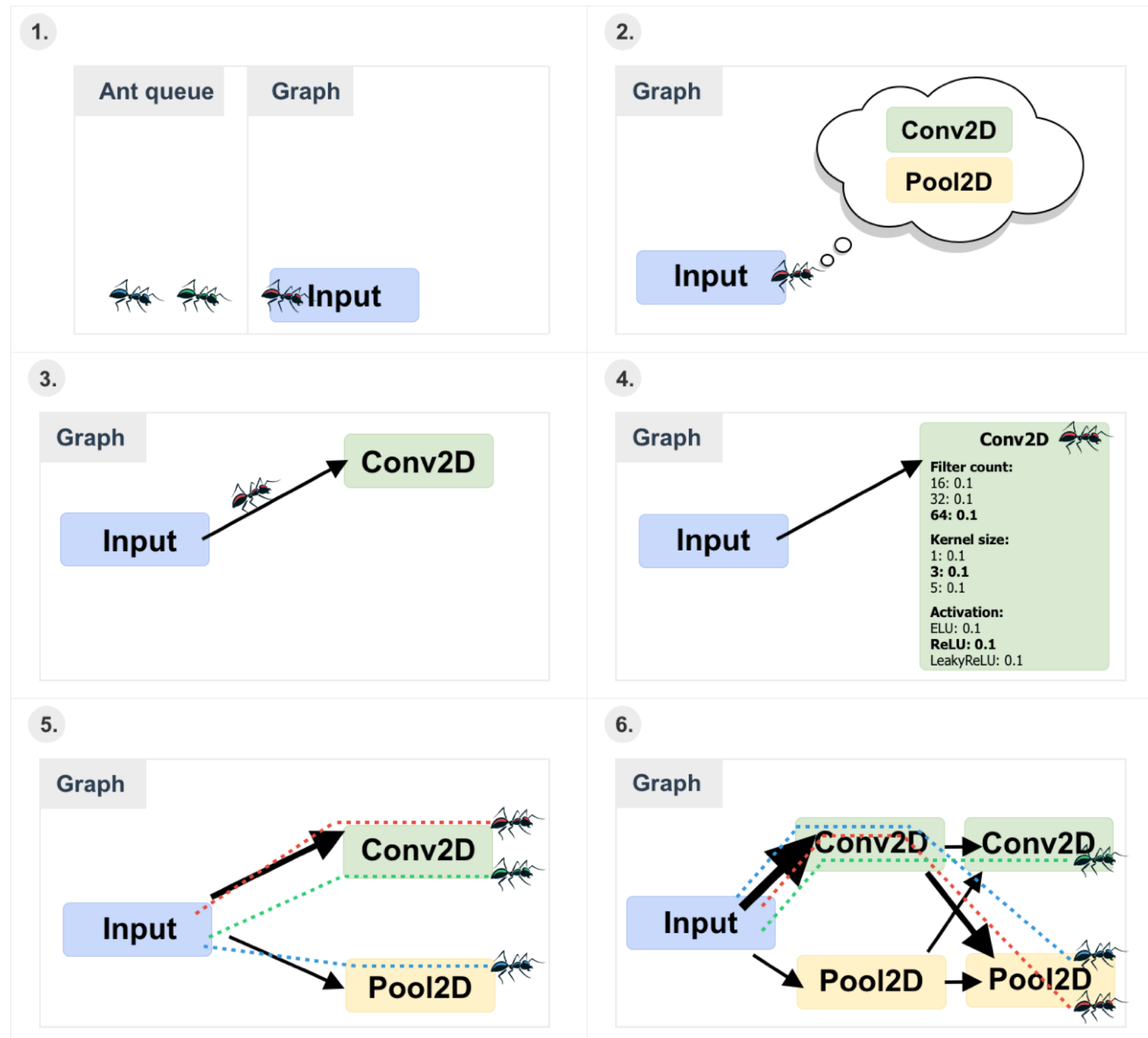


DeepSwarm

Neural Architecture Search

ANT COLONY OPTIMIZATION

DEEP-SWARM



ANT COLONY OPTIMIZATION

SOME VARIANTS

- **Ant Colony Systems (ACS)**: differs from AS in these aspects:
 - Different transition rule: biases the search towards nodes connected by short links and with a large amount of pheromone → stronger exploitation
 - Different pheromone update rule: only the best ants are allowed to update the pheromone concentrations on the links of the global-best path
 - Local pheromone updates with adaptive pheromone evaporation (i.e., adaptive ρ)
 - Lists of candidate nodes (sorted by increasing distance w.r.t. focal node) are used to favor specific nodes when the solution is constructed
- **Max-Min AS (MMAS)**: differs from AS in these aspects:
 - Different pheromone update rule: only the best ants are allowed to update the pheromone concentrations on the links of the global-best path
 - Pheromone intensities are restricted within the interval $[\tau_{\min}, \tau_{\max}]$
 - Initially, pheromones are set to the max allowed value $\tau_{\max}$ (→ exploration)
 - If stagnation is detected, pheromones are set to the max allowed value $\tau_{\max}$ (→ restart)
 - A pheromone smoothing mechanism is used (differences between high and low pheromone concentrations are reduced, such that low-pheromone links can be selected more probably)

ANT COLONY OPTIMIZATION

SOME VARIANTS

- **Elitist Pheromone Updates:** the best ants add pheromone proportional to their paths' quality (to guide the search towards constructing a solution to contains links from the current best routes)
- **Fast Ant System (FANT):** uses only one ant, same transition rule as AS (with $\beta=0$). A different pheromone update rule is applied which does not make use of any evaporation.
- **Antabu:** adapts AS to include a local search based on Tabu Search, uses a modified update rule.
- **AS-Rank:** differs from AS in these aspects:
 - Different pheromone update rule: only the best ants are allowed to update the pheromone concentrations on the links of the global-best path
 - It uses elitist pheromone updates (pheromone update rule based on a ranking of the ants)

ANT COLONY OPTIMIZATION

CONTINUOUS ANT COLONY OPTIMIZATION (CACO)

- A continuous space problem is mapped into a graph search problem
- CACO performs a bi-level search:
 - A **local search** component to exploit “good” regions
 - A **global search** component to explore “bad” regions

CACO pseudocode

Create n_r regions and set $\tau_i(0) = 1$, $i = 1, \dots, n_r$;

repeat

 Evaluate fitness, $f(\mathbf{x}_i)$, of each region;

 Sort regions in descending order of fitness;

 Send 90% of n_g global ants for mutation;

 Send 10% of n_g global ants for trail diffusion;

 Update pheromone and age of n_g weak regions;

 Send n_l ants to probabilistically chosen good regions;

 Do local search to exploit good region;

until *stopping condition is true* ;

Return region $\mathbf{x}_i$ with best fitness as solution;

ANT COLONY OPTIMIZATION

CONTINUOUS ANT COLONY OPTIMIZATION (CACO)

- **Initialization:**

1. Search is performed by n_k ants: n_l ants for local search + n_g ants for global search
2. Create n_r regions, where:
 - Each region represents a point in the continuous search space
 - If $\mathbf{x}_i$ denotes the i -th region, then $x_{ij} \sim U(x_{min,j}, x_{max,j})$
3. Set pheromone for each region to 1.0

Local search pseudocode

```
for each local ant do  
    if region with improved fitness is found then  
        Move ant to better region;  
        Update pheromone;  
    end  
    else  
        Increase age of region;  
        Choose new random direction;  
    end  
    Evaporate all pheromone;  
end
```

ANT COLONY OPTIMIZATION

CONTINUOUS ANT COLONY OPTIMIZATION (CACO)

- **Local search:** Each ant k of the n_l local ants selects a region $\mathbf{x}_i$ based on the probability:

$$p_i^k(t) = \frac{\tau_i^\alpha(t) \eta_i^\beta(t)}{\sum_{j \in \mathcal{N}_i^k} \tau_j^\alpha(t) \eta_j^\beta(t)}$$

- The ant moves a distance away from the selected region, $\mathbf{x}_i$, to a new region, $\mathbf{x}'_i$, using:

$$\mathbf{x}'_i = \mathbf{x}_i + \Delta \mathbf{x}$$

where $\Delta x_{ij} = c_i - m_i a_i$, with c_i and m_i user-defined parameters, and a_i the age of region $\mathbf{x}_i$

- **Global search:** Most ants perform mutation to produce new regions. For each variable x'_{ij} of the offspring, choose a random “bad” region $\mathbf{x}_i$. Let $x'_{ij} = x_{ij}$, with probability P_c .

Mutate the offspring $\mathbf{x}'_i$ as follows:

$$x'_{ij} \leftarrow x'_{ij} + N(0, \sigma^2), \quad \sigma \leftarrow \sigma_{max} (1 - r^{(1-t/n_t)^{\gamma_1}})$$

where $r \sim U(0,1)$, σ_{max} is the maximum step size, t is the current iteration, n_t is the maximum number of iterations, and γ_1 controls the degree of nonlinearity

- Trail diffusion: randomly select two weak regions as parents, $\mathbf{x}_i$ and $\mathbf{x}_l$, then apply arithmetic crossover:

$$x'_j = \gamma_2 x_{ij} + (1 - \gamma_2) x_{lj} \quad \text{where } \gamma_2 \sim U(0,1)$$

ANT COLONY OPTIMIZATION

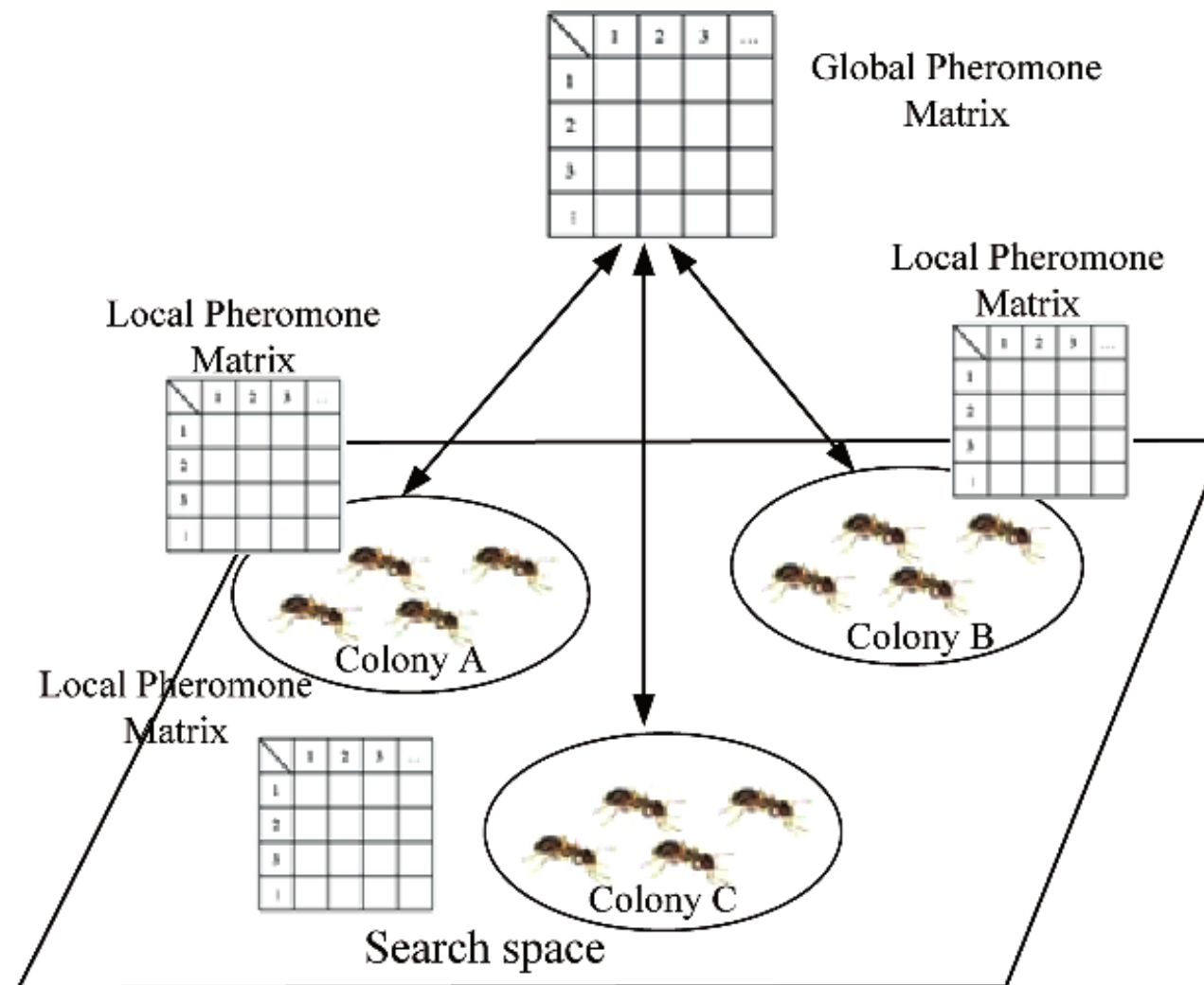
CONTINUOUS ANT COLONY OPTIMIZATION (CACO)

- **Region age and pheromone update**
 - The age of a region indicates the “weakness” of the corresponding solution
 - If $\mathbf{x}'_i$ does not have a better fitness than x_i , the age of $\mathbf{x}_i$ is incremented
 - If $\mathbf{x}'_i$ has a better fitness, the position vector of the i-th region is replaced with the new vector $\mathbf{x}'_i$
 - The direction on which an ant moves remains the same if a region of better fitness is found
 - If the new region is less fit, a new direction is randomly chosen
 - Pheromone is updated by adding an amount to each τ_i proportional to the fitness of that region

ANT COLONY OPTIMIZATION

MULTI-OBJECTIVE ANT COLONY OPTIMIZATION

- Two main methods:
 - **Multi-pheromone approach** → one pheromone for each objective
 - **Multi-colony approach** → one colony for each objective



ANT COLONY OPTIMIZATION

CONCLUDING REMARKS

- ACO is especially suited for solving hard combinatorial (graph-based) optimization problems.
- Artificial ants implement a randomized construction heuristic which makes probabilistic decisions.
- The a cumulated search experience is taken into account by adapting a pheromone trail.
- ACO shows great performance with dynamic problems, like network routing.
- Modern versions of ACO include a local search (between the local and global update).
 - ▶ State-of-the-art results on very large (>100k cities) instances of TSP!

Questions?